

Compiler requirements

Use Microsoft Visual Studio to compile SystemC models. Microsoft Visual Studio 2019 or higher required..

SystemC distribution

Use the systemc-2.3.1 SystemC distribution (or higher).

Compiling the SystemC distribution (`SystemC.lib`)

Use the MSVC project file in the distribution (`<sc_home>\msvc10\SystemC`)

- C++/Code generation: Multi-threaded DLL (/MDd)
- Extra cmd line option: /vmg
- Build and the result will be in:
`<sc_home>\msvc10\SystemC\$(Configuration)\SystemC.lib`

Compiling your model

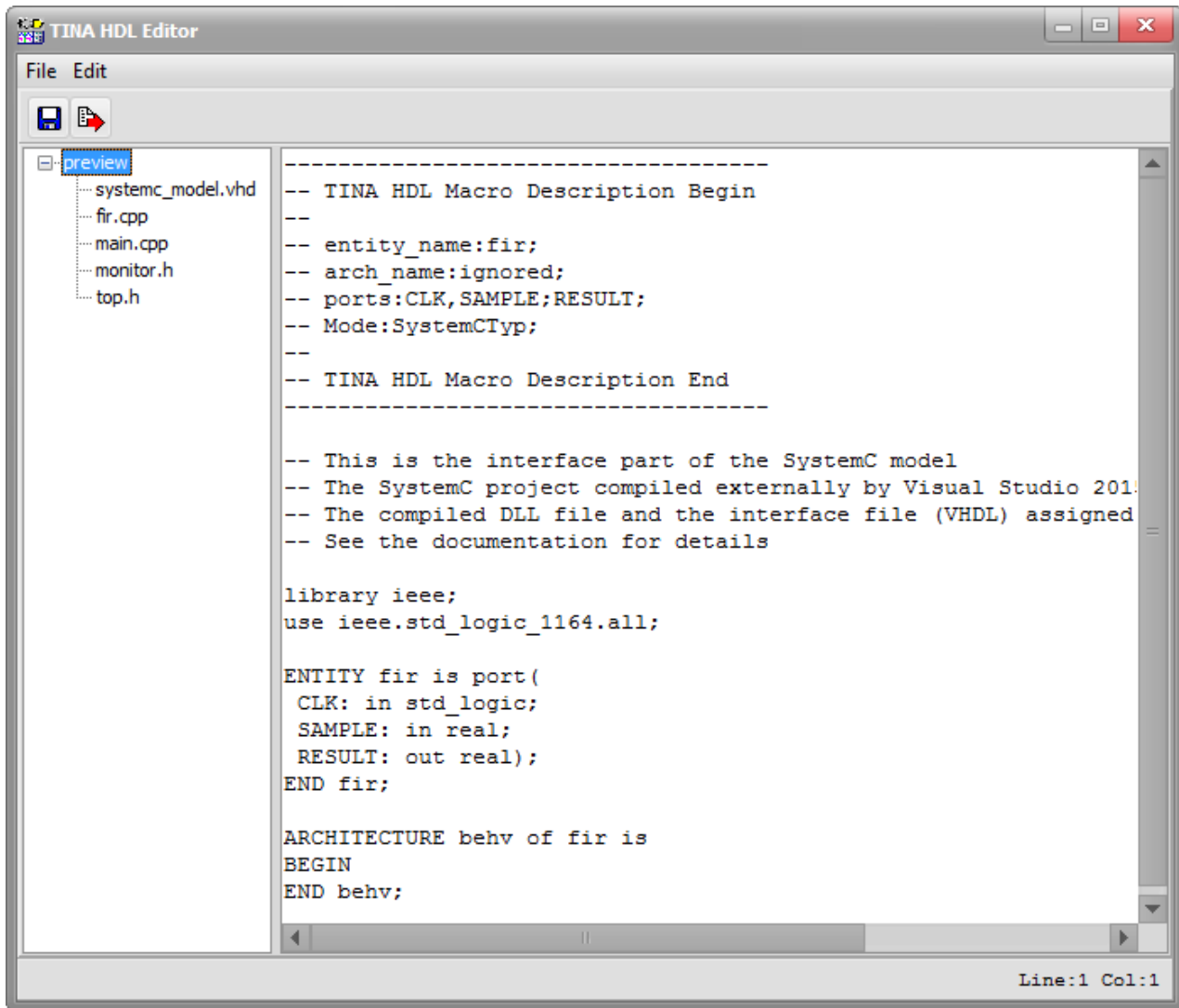
Use the project template in `<TINADir>\Examples\SystemC\systemc_model.zip` (`systemc\systemc_model.vcxproj`).

Open Visual Studio and open the project file. Open the property manager (View/Other windows/Property manager) and select the “Macros” entry, select Common properties/User macros. Change the SC_HOME macro where you extracted the SystemC distribution.

Creating the macro in TINA

Create the VHDL interface file (described below). Name the interface file same as the model dll file, for example `sc_model.vhd`, `sc_model.dll`. Copy this files to the same directory, then select Tools/New Macro Wizard, select From File, select `SystemC executable (*.dll)`. Select *Auto generate shape*. Rearrange the pins if needed by right clicking on the shape and selecting *Edit Symbol...* after creating the macro. If you copy some `*.c` and `*.h` files to this directory this files will be added to the macro for previewing purpose.

After creating the macro double click on the macro and select `Enter Macro`. The macro will look like this



Modelling requirements

In `sc_main` use dynamic allocation for the top level module. Return with 0 after creating the top instance.

```

int sc_main(int argc, char* argv[])
{
    top* TOP = new top("TOP");

    return 0;
}

```

The SystemC macro consists of a VHDL file and a SystemC dll file. The VHDL file is an interface file only. The SystemC top level module must contain the same signals as in the VHDL interface module and in the same order.

Example top level SystemC file (counter)

```

SC_MODULE(top)
{
    sc_signal<sc_logic> CLK, CLEAR, QA, QB, QC, QD;

    counter U1;

    SC_CTOR(top): U1("U1")
    {
        U1.CLK(CLK); U1.CLEAR(CLEAR); U1.QA(QA); U1.QB(QB); U1.QC(QC); U1.QD(QD);
    }
}

```

```

    }
};

```

The most important part of the counter SystemC model

```

#ifndef counterH
#define counterH

#include "monitor.h"

SC_MODULE(counter)
{
    sc_in<sc_logic> CLK, CLEAR;
    sc_out<sc_logic> QA, QB, QC, QD;
    int value;

    monitor MON;

    void proc()
    {
        double t = sc_get_curr_simcontext()->time_stamp().to_seconds();
        if (CLEAR.event() && CLEAR == SC_LOGIC_0 ) {
            value = 0;
        }
        else if (t > 0 && CLK.event() && CLK == SC_LOGIC_1) {
            value++;
            if (value == 10) value = 0;
        }

        sc_lv<4> sclv_value(value);
        QA = sclv_value[0];
        QB = sclv_value[1];
        QC = sclv_value[2];
        QD = sclv_value[3];
    }

    SC_CTOR(counter): MON("MON")
    {
        value = 0;
        SC_METHOD(proc);
        sensitive << CLK.value_changed() << CLEAR;
        MON << QA << QB << QC << QD;
    }
};
#endif

```

Example VHDL file (counter)

```

library ieee;
use ieee.std_logic_1164.all;

ENTITY counter is port(
    CLK: in std_logic;
    CLEAR: in std_logic;
    QA, QB, QC, QD: out std_logic);
END counter;

```

```

ARCHITECTURE behv of counter is
BEGIN
END behv;

```

If you have signals in the `sc_main` you have to declare it dynamic.

Supported top level port types in mixed mode

`sc_bit`, `sc_logic`, `bool`, `double`

TINA Analysis setup

Select a proper time step in Analysis/Set Analysis Parameters/TR maximum time step

Examples

Counter example

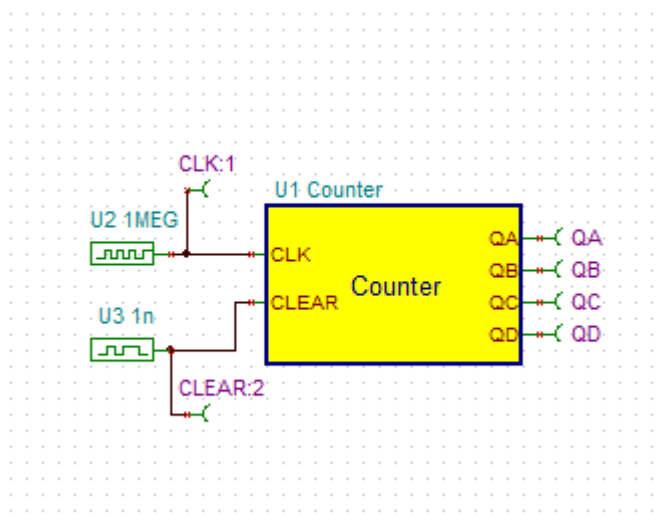
Use the project template in `<TINADir>\Examples\SystemC\systemc_model.zip` (`systemc\systemc_model.vcxproj`) . Extract this zip file (`<sc_model>`)

Open Visual Studio and open the project file. Open the property manager (View/Other windows/Property manager) and select the “Macros” entry. Change the `SC_HOME` macro where you extracted the `systemc_model.zip` file.

Copy the `<sc_model>\systemc_model\Examples\counter\top.h` to `<sc_model>\systemc_model` and `<sc_model>\systemc_model\Examples\counter\monitor.h` to `<sc_model>\systemc_model`. Rebuild the project.

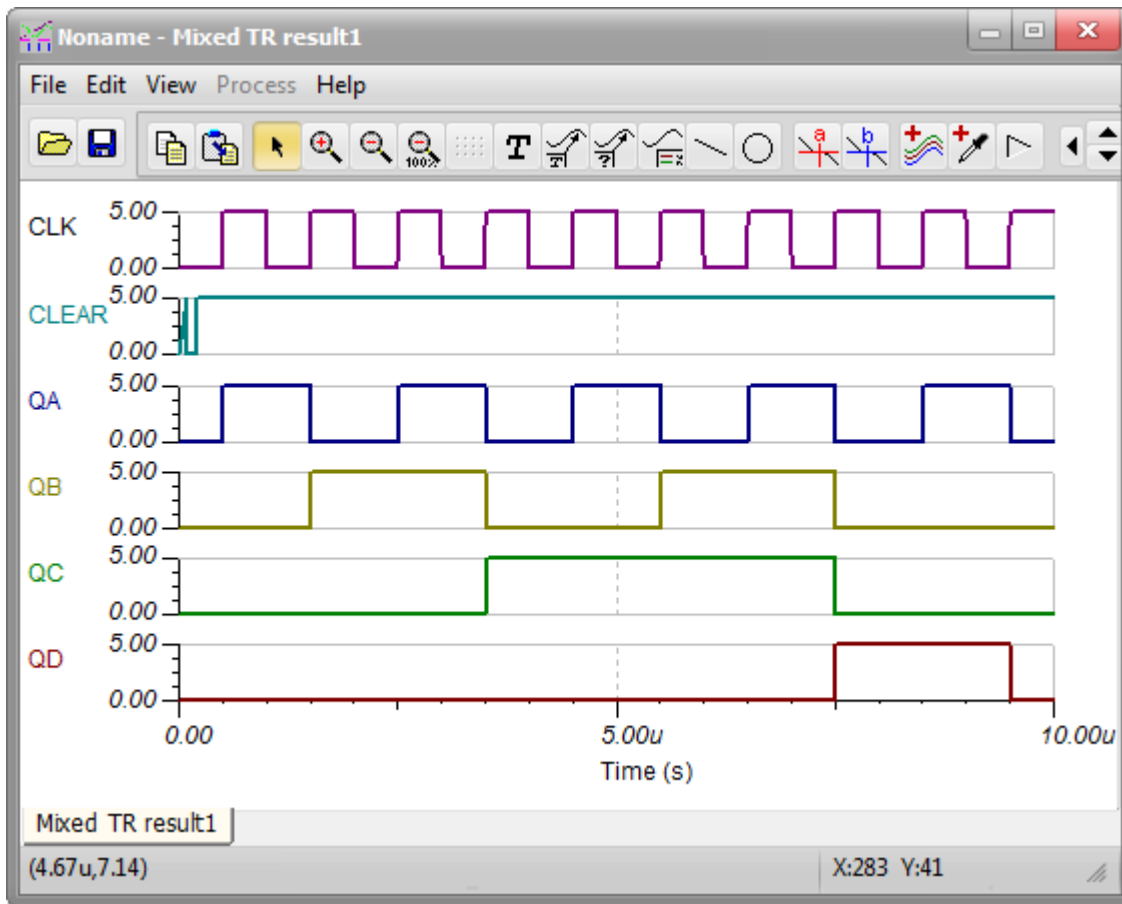
Copy the `<sc_model>\systemc_model\Examples\counter\systemc_model.vhd` and `<sc_model>\$(Configuration)\systemc_model.dll` to a directory (for example `d:\Temp`).

In TINA Open the `<TINADir>\Examples\SystemC\counter.tsc`, delete the counter macro.



Now select Tools/New Macro Wizard... Type Counter in the Macro Name field, select From File, press the folder icon. In the dialog select Files of type “SystemC executable” and search for the previously copied `systemc_model.dll`.

Insert the new macro to previously place of the previously deleted macro. Run Transient. The result is the following.



Microcontroller example

This examples assumes you have the Microchip XC8 compiler installed.

Now let's make a new microcontroller that is currently not in TINA (PIC16LF1906).

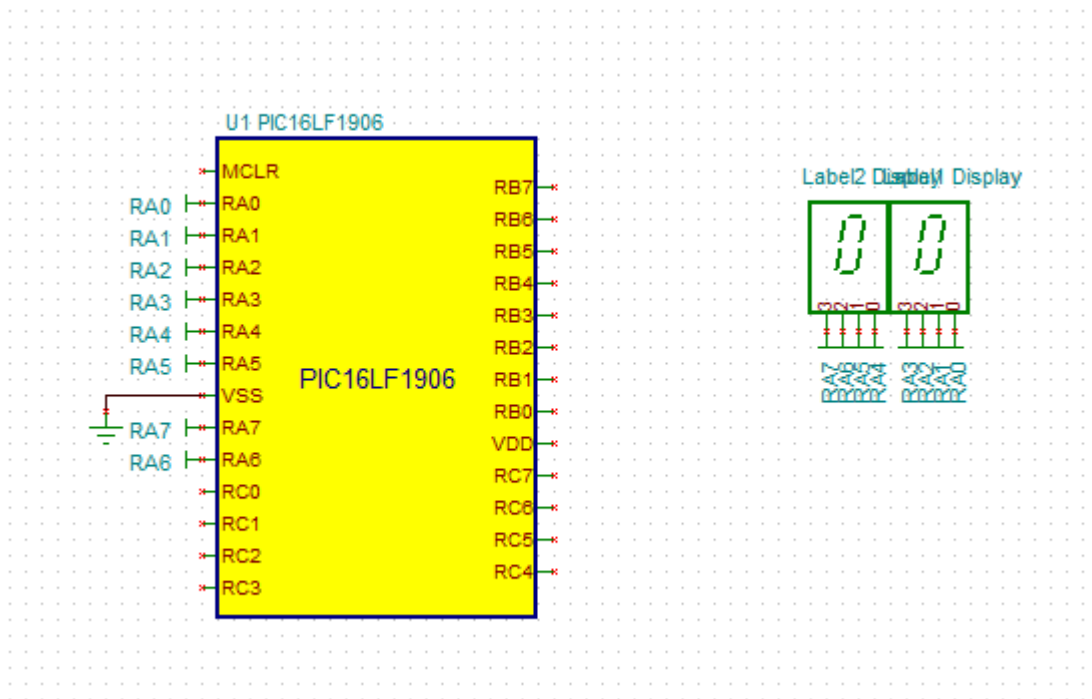
At first build the new model.

Copy the <sc_model>\systemc_model\Examples\pic16lf1906\top.h to <sc_model>\systemc_model and <sc_model>\systemc_model\Examples\pic16lf1906\monitor.h to <sc_model>\systemc_model. Rebuild the project.

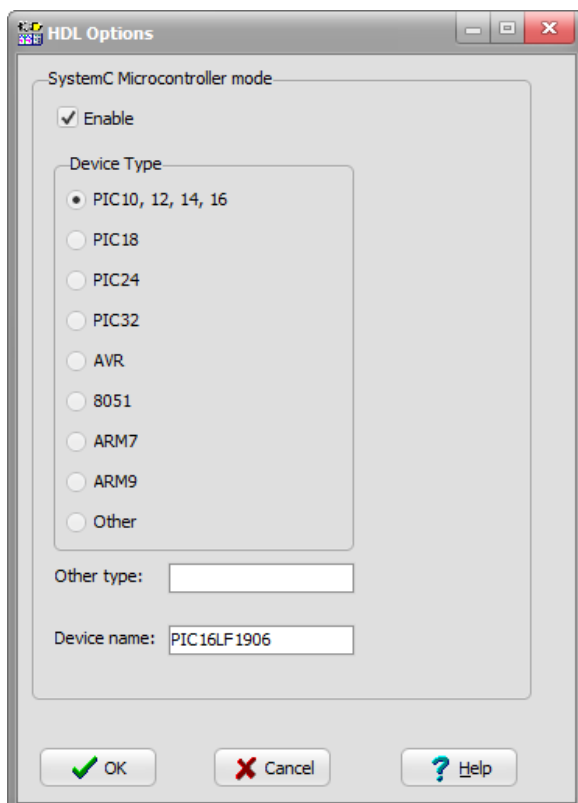
Copy the <sc_model>\systemc_model\Examples\pic16lf1906\systemc_model.vhd and <sc_model>\\$(Configuration)\systemc_model.dll to a directory (for example d:\Temp).

Flasher example

In TINA Open the <TINADir>\Examples\SystemC\PIC16LF1906 sc flasher.TSC, delete the PIC macro.



Now select Tools/New Macro Wizard... Type PIC16LF1906 in the Macro Name field, select From File, press the folder icon. In the dialog select Files of type "SystemC executable" and search for the previously copied systemc_model.dll. Select the "Options" button and check Enabled, check device type PIC16, enter "device name" PIC16LF1906, press OK.



Insert the new macro to previously place of the previously deleted macro.

Now assign a C code to the macro. The code implements a counter. The counter value will be displayed on PORTA.

Click on the PIC macro and press the “...” field in the ASM-Code field. Select C code and copy the content of the flasher.c (<sc_model>\systemc_model\Examples\pic16lf1906\flasher.c) as a new file in the editor, press Make, press OK.

Press the TR button to Run interactive transient.

The flasher C code. This C code will run on the SystemC microcontroller model.

```
#define _XTAL_FREQ 1000000

#include <xc.h>
#include <stdio.h>
#include <stdlib.h>

/*
 *
 */
int main(int argc, char** argv)
{
    unsigned char a, b, op, res;

    TRISA = 0x00;
    res = 0;

    while (1) {

        PORTA = res;
        res++;

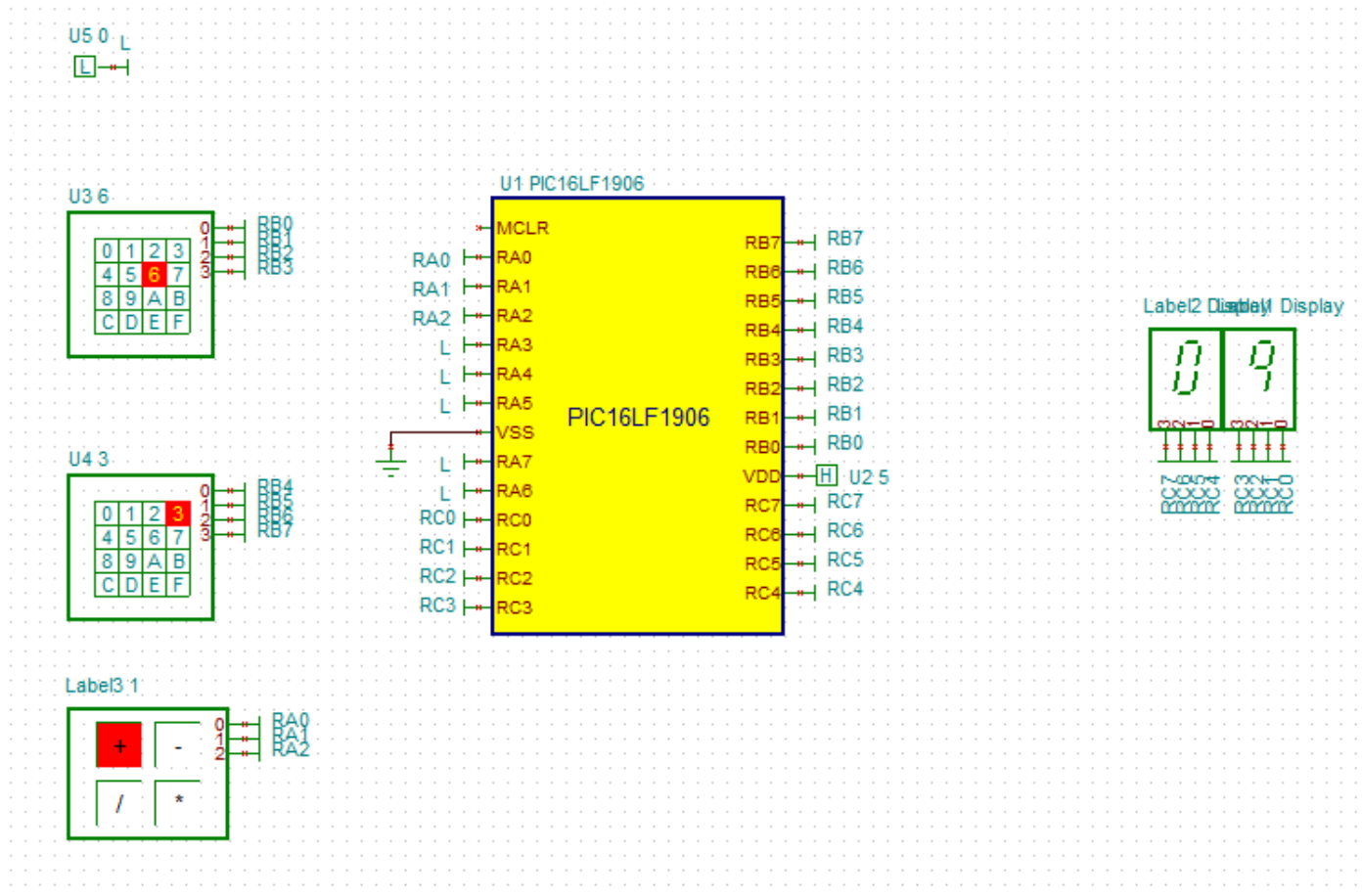
    }

    return (EXIT_SUCCESS);
}
```

Calculator example

Now let's make a simple calculator.

In TINA Open the <TINADir>\Examples\SystemC\PIC16LF1906 sc calc.TSC, delete the PIC macro.



The steps are the same to the previous process except now we assign the calc.c code.
(alternatively you can copy paste the macro from the previous circuit and change the C code)

Press the TR button to Run interactive transient.

Press the numbers on the keypad and +,-,/,* signs. The hex displays will show the result.

The calculator C code. This C code will run on the SystemC microcontroller model.

```
#define _XTAL_FREQ 1000000

#include <xc.h>
#include <stdio.h>
#include <stdlib.h>

/*
 *
 */
int main(int argc, char** argv)
{
    unsigned char a, b, op, res;

    TRISA = 0xFF;
    TRISB = 0xFF;
    TRISC = 0x00;

    while (1) {

        a = PORTB & 0x0F;
        b = (PORTB & 0xF0) >> 4;
```

```

op = PORTA & 0x0F;

if (op == 1)
res = a+b;
else if (op == 2)
res = a-b;
else if (op == 3)
res = a/b;
else
res = a*b;

PORTC = res;

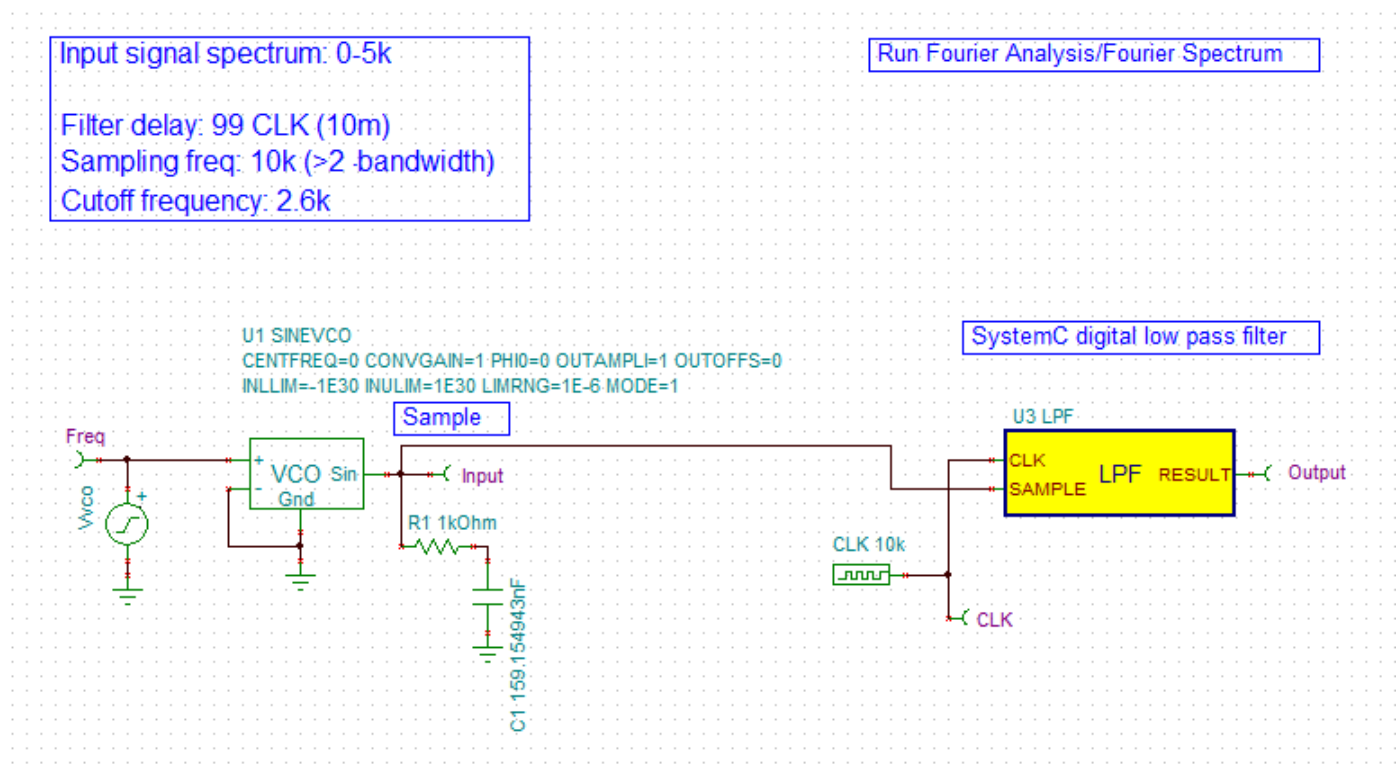
}

return (EXIT_SUCCESS);
}

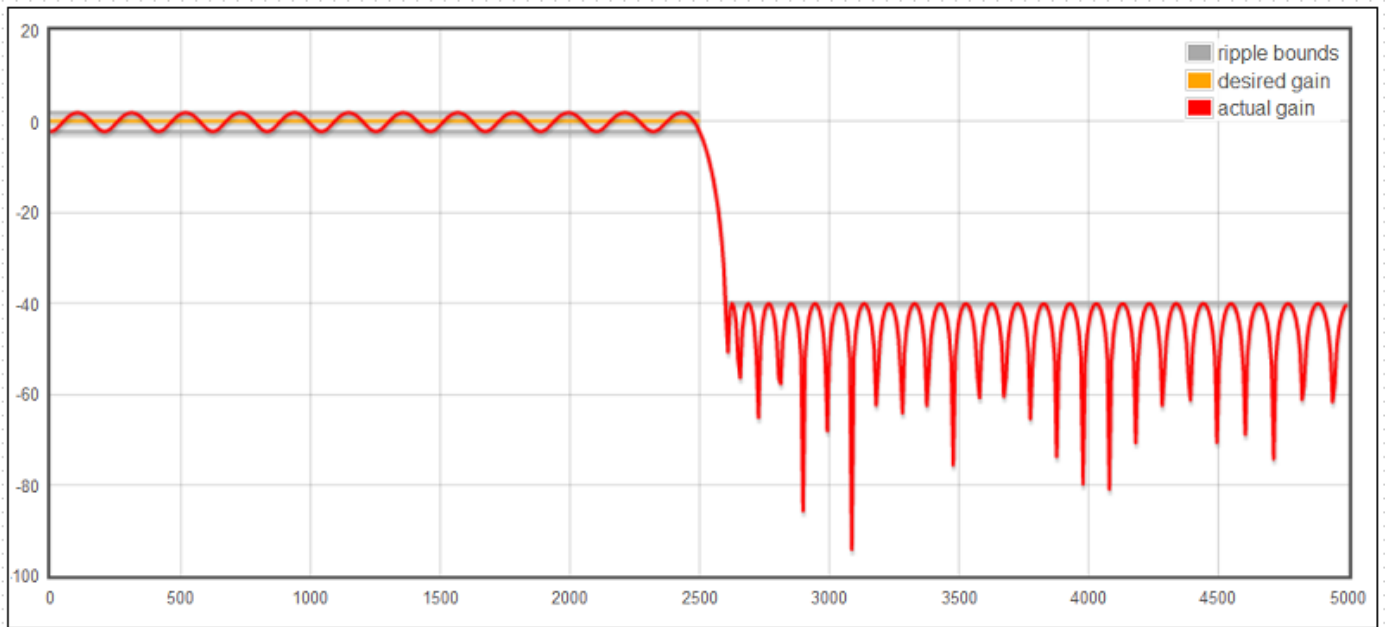
```

Low-pass filter example

Open the <TINADir>\Examples\SystemC\sc lpf.TSC example.



The filter characteristics

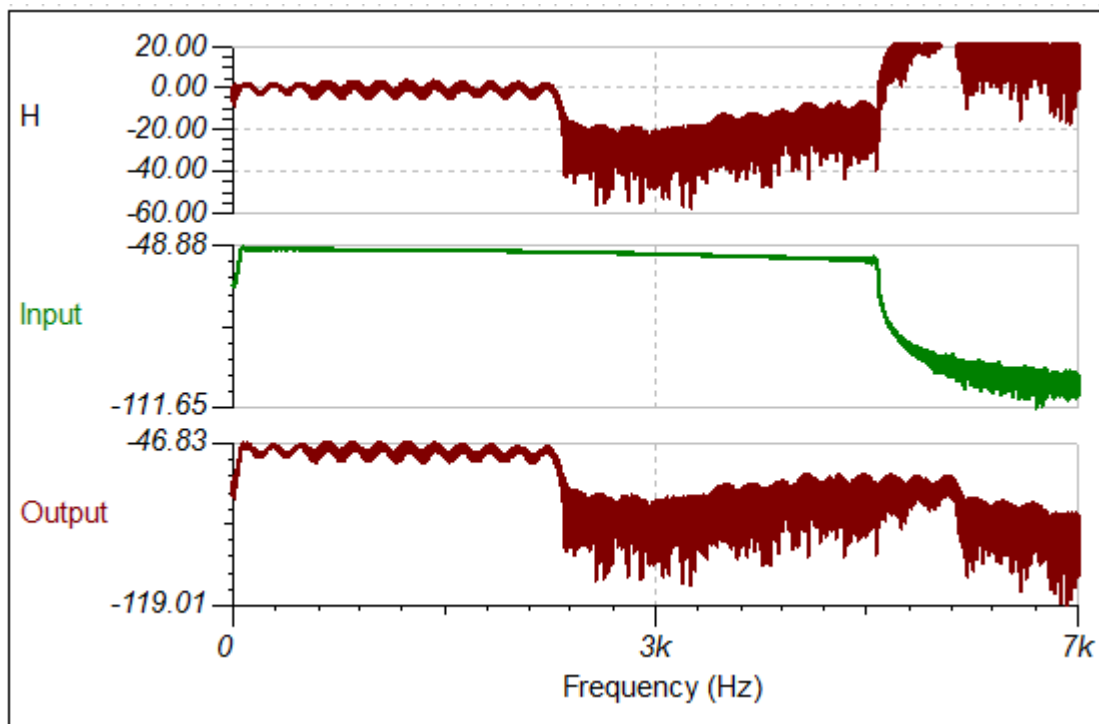


Run Analysis/Fourier Spectrum...

Press the  button in the diagram window. The Post-processor will appear. Now draw the transfer function.

Press the *More* button. Type *Output(s)/Input(s)* in the Line Edit. Type *H* in the new function name. Press *Create*. Press *OK*. Select *View/Separate curves*.

The result is the following



The most important part of the SystemC macro:

```
void fir::proc()
```

```

{
    double u, y;

    if (CLK.read() == SC_LOGIC_1) {
        sc_logic sc_val;

        // CALC
        x[0] = SAMPLE; y = 0;
        for (int k=0; k<M; k++) {
            y += b[k]*x[k];
        }

        // SHIFT
        for (int k=M-1; k>=1; k--)
            x[k] = x[k-1];

        // SET VALUE
        RESULT = y;

        n++;

    }
}

```

You can find this example in the Examples/Fir folder of the systemc_model.zip.

How to create your own microcontroller

You can create your own microcontroller model based on our code (<TINADir>\Examples\SystemC\systemc_model.zip). Our code implements the PIC16LF1906 PIC microcontroller SystemC model core (no peripherals).

Now let's make a new microcontroller model (PIC16LF1907 40 PIN PDIP). This new PIC model has two new port (PORTD, PORTE).

At first modify the VHDL interface file. Change the entity name to PIC16LF1907 and add ports *RD0-7*, *RE0-3*. You can find the modified file in <sc_model>\systemc_model\Examples\pic16lf1907\systemc_model.vhd.

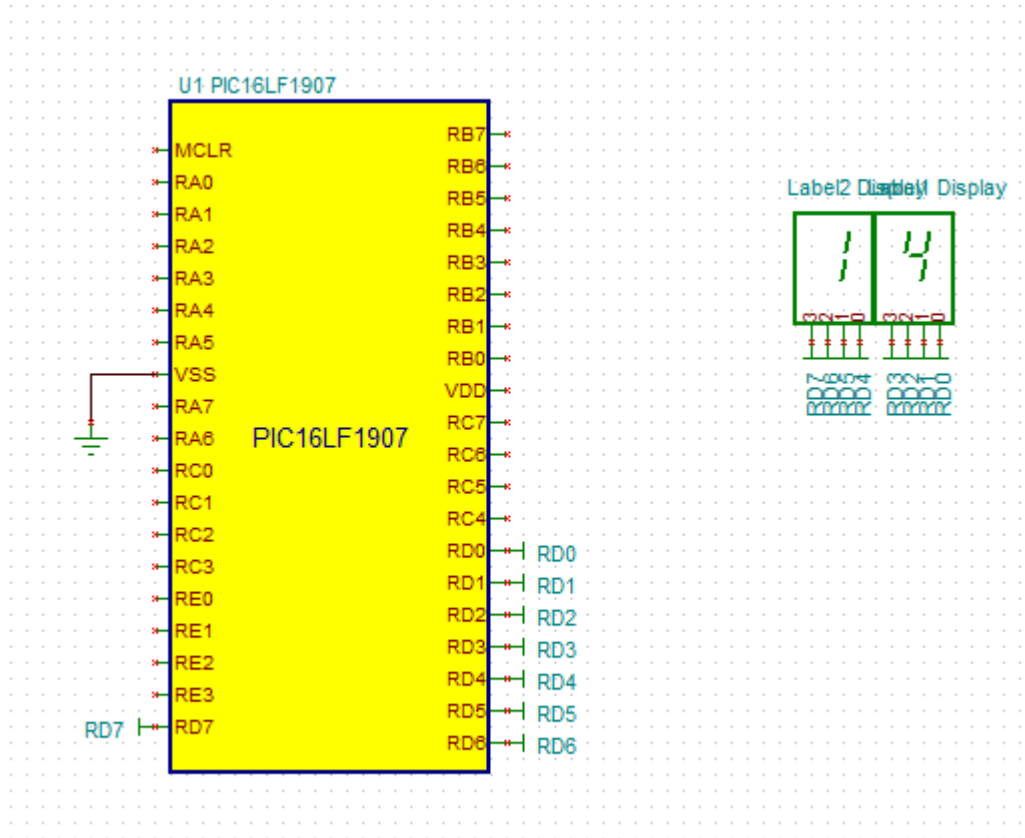
Next add the port handling to the SystemC model. Add the new ports and sensitive statements to *monitor.h*. Add the new top level signals in *top.h*. Add the new *PORTD*, *PORTE* defines to *pic.h*, add the new pin declarations, add the new sensitivity to *MON*. In *PicSimulator.h* add the new defines *TRISD*, *TRISE*. In *PicSimulator::SetDevice* modify the *SetPinLayout* call to reflects the new pins. In *pic::OnChangePinData* add the PORTC-PORTE cases, add pin assignments *RD0-7*, *RE0-3*, in *pic::IsTRISAddress* add new cases *TRISD*, *TRISE*

The changes you will find in <sc_model>\systemc_model\Examples\pic16lf1907.

Now let's test the new microcontroller model.

Compile the model with Visual Studio. Copy the compiled model (systemc_model.dll) and the the interface file (systemc_model.vhd) to a directory. Create your new macro as described previously.

Now let's test the new port functionality. The flasher.c in <sc_model>\systemc_model\Examples\pic16lf1907 is modified, it writes the data to the new port, PORTD. Assign the C code as described previously. Set a small time step (1u) in Analysis/Set Analysis Parameters/TR maximum time step. Test by pressing the TR button.



TINA SystemC microcontroller API

The top module called *pic*. Ports declared by *sc_inout* (pins) and *sc_lv* (ports itself). Clock implemented by *clock_proc*, the *simulator->iNanoPerClock* member updated from TINA by the frequency component property. *Pic::end_of_elaboration* called by the SystemC engine at the end of the design elaboration, this method calls *pic::InitSimulator* which initializes the *simulator*.

The main simulator class type called *McuSuperSimulator*, *McuSuperSimulator::OnTimeChange* called on each clock rising edge change.

The following methods are important and need to be implemented:

- *pic::OnChangePinData*: called on I/O changes, updates the port I/O signals
- *PicSimulator::SetPinDataX*: updates the port I/O signals (they are input changes, sets by *sc_pli_ad_interface* which is called from TINA when an input changes on a mcu pin)
- *pic::AddDDRChange*: monitors data direction changes (*sc_pli_mcu_get_ddr_change_info*, *sc_pli_mcu_clear_ddr_changes* will be called from TINA)